

≡ Hide menu

About this Course

Key Components of Llamafile

Developing Portable CLI with Cosmopolitan

✔ **Reading:** Key Terms
1 min

✔ **Video:** Building a phrase generator with cosmopolitan
3 min

📅 **Lab:** Portable CLI
10 min

📖 **Reading:** Bash Phrase Generator
5 min

✔ **Graded Assignment:** Quiz- Portable CLI with Cosmopolitan
Submitted

📖 **Reading:** Lesson Reflection
5 min

Running Llamafile

Conclusion and Next Steps

Portable CLI

🔗 Launch lab

Instructions

Lab Goal

Apply concepts from lesson by modifying, compiling, and running a reusable phrase generation program using Cosmopolitan

```
1 //bin/cosmocc -o phrase-gen phrase-gen.c
2 ../phrase-gen -c 5 -p "Hello there"
3
4 #include <stdio.h>
5 #include <string.h>
6 #include <stdlib.h>
7
8 // Repeat phrase N times
9 void phrase_generator(int count, char* phrase) {
10     for (int i = 0; i < count; i++) {
11         printf("%s\n", phrase);
12     }
13 }
14
15 int main(int argc, char* argv[]) {
16
17     // Default values
18     int count = 1;
19     char* phrase = "Hello world";
20
21     // Parse command line arguments
22     for (int i = 1; i < argc; i++) {
23         if (strcmp(argv[i], "--count") == 0 || strcmp(argv[i], "-c") == 0) {
24             count = atoi(argv[i+1]);
25         }
26         else if (strcmp(argv[i], "--phrase") == 0 || strcmp(argv[i], "-p") == 0) {
27             phrase = argv[i+1];
28         }
29     }
30
31     // Generate phrases
32     phrase_generator(count, phrase);
33
34     return 0;
35 }
```

Run

Reset

Hello world

Lab Description & Steps

1. Analyze existing phrase-gen.c program and identify key sections
2. Add support for a new "--uppercase" CLI argument
3. Handle argument parsing and update phrase_generator()
4. Compile updated program with cosmocc compiler
5. Test running with varied counts, phrases, and new upper-casing

Reflection Questions

1. How did supporting a new argument require updating multiple functions?
2. What was needed to successfully handle command line arguments?
3. How was support for arbitrary phrase repetition implemented?
4. Why is modular design with separate parse/process/display functions useful?
5. How did cosmocc facilitate building a portable command line application?

Challenge Exercises

1. Simplify argument checking using a loop over argv array
2. Support repeating multiline phrases as input
3. Print number of characters for each output repetition
4. Fetch dynamic phrases from a file or API endpoint
5. Add additional CLI options like separator strings between repeats

Share My Work

Generate or update a read-only copy of your lab. Learn more

Generate/Update shared link

👍 Like 👎 Dislike 📄 Report an issue